INFORMATION SECURITY MIDTERM THEORY PROJECT • TOPIC 13

Security in Client-Server Android Applications

Android Permissions, Intent Security, and Local Data Protection

ACADEMIC DETAILS

Course: Introduction to Information Security

Instructor: Huynh Ngoc Tu, Ph.D

Class ID: N05 | **Group ID: G02**

PROJECT TEAM (GROUP G02)

Ngô Lâm Tiến (Leader) | **ID:** 524H0195

Nguyễn Công Thành | **ID:** 524H0191

Võ Huỳnh Duy Tân | **ID:** 524H0187

THREAT LANDSCAPE & ATTACK SURFACE

Why Android Security Matters

The Multi-Layered Attack Surface of Mobile Client-Server Systems

End User
Device Owner

→

Android Client
Application Logic

→

Storage / IPC
SQLite & Intents

→

HTTPS / Transit
Network Channel

→

Backend Server
Cloud APIs & DB

1. Permissions Boundary

Gates sensitive device hardware, location, contacts, and privileged system APIs.

Vulnerability: Over-privilege, requesting unnecessary dangerous permissions, and failing to handle runtime revocations.

Governs Device Capabilities

2. IPC & Intent Boundary

Governs message passing between distinct application processes and components.

Vulnerability: Exported components blindly trusting unvalidated Intent extras and forwarding nested payloads.

Mediates Inter-App Trust

3. Storage-at-Rest Boundary

Isolates private SQLite databases, cached tokens, and preferences via Linux UIDs.

Vulnerability: Plaintext data exposed via Android backups, rooted devices, or exported ContentProviders.

Protects Local Assets

4. Network Transit Boundary

Secures client-server REST API payloads and authentication headers over HTTPS.

Vulnerability: Man-in-the-Middle (MITM) traffic decryption if user-installed root CAs are trusted without pinning.

Secures Remote Transit

KEY TAKEAWAY: Android security is not a single perimeter — it requires defense-in-depth across all four boundaries.

SYSTEM ARCHITECTURE

Android Security Foundation

Kernel-Level Sandboxing & Component-Based Isolation

Linux Application Sandbox

Per-App Linux UID: Each app is assigned an immutable Linux UID (e.g. `u0_a142`) at installation.

Process Isolation: Applications execute inside dedicated processes running isolated Android Runtime (ART) instances.

Kernel File Security: Internal data directories (`/data/data/<pkg>`) are assigned strict `0700` file permissions.

First Security Boundary: Prevents ordinary apps from directly reading another app's memory or private disk files.

Core Principle: Multi-tenant Linux OS model applied to mobile apps.

The 4 Core Components

Activity: Visual UI screens handling rendering and direct touch interaction.

Service: Background worker performing long-running tasks without UI (network sync, music).

BroadcastReceiver: Asynchronous listener reacting to system-wide or app broadcasts.

ContentProvider: Structured data management interface abstracting SQLite access for IPC.

Every component has distinct lifecycle and IPC semantics.

AndroidManifest.xml Authority

Authoritative Blueprint: Root XML file declaring all app components, permissions, and security parameters.

Component Exposure: The critical `android:exported` attribute dictates public accessibility to external apps.

Permission Binding: Enforces required capabilities (`<uses-permission>`) and custom signature security gates.

Security Sensitivity: A single misconfiguration here can compromise the entire sandbox boundary!

Single authoritative security contract parsed at install time.

ACCESS CONTROL & GOVERNANCE

Permission Model & AndroidManifest.xml

Granular Access Governance & The Principle of Least Privilege

Normal (Install-Time)

LOW RISK

Minimal risk (`INTERNET`, `ACCESS_NETWORK_STATE`). Auto-granted upon installation without prompting user.

Dangerous (Runtime)

HIGH RISK

Accesses sensitive personal data (`CAMERA`, `READ_CONTACTS`, `FINE_LOCATION`). Requires explicit user prompt approval at runtime.

Signature

VENDOR PROTECTED

Highest security. Granted automatically ONLY if requesting app is signed by the exact same developer certificate.

Special / Privileged

SYSTEM LEVEL

System-level capabilities (`SYSTEM_ALERT_WINDOW`, `WRITE_SETTINGS`). Requires manual user activation in system settings.

The android:exported Gateway

exported="false": Component is strictly private. Only reachable by internal app logic or apps sharing the exact same Linux UID.

exported="true": Component is public to the entire OS. ANY third-party application can send Intents to launch it.

The Intent Filter Trap: Declaring an `<intent-filter>` historically defaulted exported to true! In Android 12+, developers MUST declare exported explicitly.

```
<!-- Secure Internal Component Declaration -->
<activity
    android:name=".InternalAdminActivity"
    android:exported="false" />
```

Principle of Least Privilege:
Request only the minimal permissions strictly required, and keep all internal components non-exported.

INTER-PROCESS COMMUNICATION

Android Intents & IPC Mechanics

Message-Passing Channels Crossing Security Trust Boundaries

Explicit Intent (Internal / Targeted)

Direct Binding: Designates exact target class: `Intent(context, TargetActivity::class.java)`.

Delivered exclusively to the specified component within the application.

Highly Secure: No dynamic resolution; immune to third-party interception.

Implicit Intent (Public / Dynamic)

Dynamic Resolution: Specifies Action (e.g. `ACTION_VIEW`) and Data URI, but no target class.

Package Manager matches against declared `<intent-filter>` tags.

Security Risk: Malicious apps can register matching filters to hijack data.

Intent Object Structure

Action

e.g. `ACTION_SEND`, `ACTION_VIEW`. Defines requested operation.

Data & Type (URI)

e.g. `content://`, `https://`. Specifies the data target.

Extras Bundle (Parcelable)

Key-value parameters. Can carry serialized **Nested Intents**!

CRITICAL SECURITY PRINCIPLE: Crossing application boundaries via Intents creates a formal Trust Boundary. Any Intent received from an external application must be treated as hostile, untrusted user input!

VULNERABILITY MECHANICS

Attack #1: Intent Injection & Redirection

Confused Deputy Exploitation via Unvalidated Intent Forwarding

1. Attacker App
Crafts malicious Intent with nested target Intent

→

2. Trust Boundary
Sent to exported Victim Proxy Activity

→

3. Vulnerability Point
Proxy extracts nested extra without validation

→

4. Execution
startActivity(next) runs internal private activity!

```
// VULNERABLE PROXY COMPONENT (Exported Activity)
class ProxyActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // 1. Attacker controls 'next_intent' in Extras
        val next = intent.getParcelableExtra<Intent>("next_intent")

        // 2. VULNERABILITY POINT: Blind forwarding!
        // Launches unexported activity with Victim's UID!
        startActivity(next)
    }
}
```

How Exploitation Works:

Attacker embeds an Intent targeting `InternalAdminActivity` (which has `exported="false"`). When `ProxyActivity` invokes `startActivity(next)`, it launches with the victim app's own UID privileges!

Technical Root Causes & Mechanics

The 'Confused Deputy' Flaw: The victim app acts as a proxy, executing privileged actions on behalf of an unprivileged attacker.

Bypassing Access Control: Internal activities set to `exported="false"` cannot be launched directly by outside apps, but CAN be launched internally by the victim app itself!

Lack of Sanitization: The victim app fails to inspect `next.component`, fails to verify destination package name, and maintains no component allowlist.

Severe Security Impact: Attacker can force execution of hidden administrative screens, steal sensitive tokens passed in result Intents, or trigger internal background services.

Severity: CRITICAL • Complete access control bypass

DATA AT REST VULNERABILITIES

Attack #2: Sensitive Data in Local Storage

Plaintext SQLite Databases & Secondary Exposure Vectors

SQLite: user_session.db

PLAINTEXT AT REST

- `id: 10482`
- `user_email: student@tdtu.edu.vn`
- `auth_token: eyJhbGciOiJIUzI1Ni... (Plaintext JWT)`
- `api_secret: sec_live_98234ab8... (Plaintext Key)`

Real-World Secondary Exposure Vectors

Exported ContentProvider: SQL injection or traversal leaks raw DB files.

Insecure Backup: `allowBackup="true"` enables `adb backup` extraction.

Compromised / Rooted OS: Root UID (UID 0) overrides sandbox isolation.

Debug Builds & Logs: `debuggable="true"` permits run-as extraction.

Access Control (Linux Sandbox)

Mechanism: Restricts file system read/write permissions at the OS layer via Linux UIDs (`0700` file mode).

Limitation: Protects only against ordinary unprivileged third-party apps under uncompromised OS conditions.

Failure Mode: Completely fails if root access, backup tools, or secondary file-reading bugs exist.

Controls WHO can open the file handle at OS level

Encryption at Rest (SQLCipher / Keystore)

Mechanism: Cryptographically encrypts 4KB SQLite pages using AES-256 before writing to storage.

Key Protection: Master keys generated and stored inside hardware-backed Android Keystore (TEE/SE).

Resilience: Protects data confidentiality even under total physical extraction or root-level compromise!

Controls WHAT can be read from raw storage bytes

RISK EVALUATION & IMPACT

Security Impact: The CIA Triad

Evaluating Systemic Harm in Mobile Client-Server Environments

CONFIDENTIALITY

SEVERITY: HIGH

Bearer Token Theft: Extraction of plaintext JWT session tokens from unencrypted SQLite databases or SharedPreferences.

Private PII Exposure: Leakage of user chat messages, contact lists, financial details, and personal records.

API Secret Leaks: Exfiltration of proprietary cloud backend API keys and cryptographic signing keys stored in client code.

Risk: Direct Account Takeover & Cloud Abuse

INTEGRITY

SEVERITY: CRITICAL

Confused Deputy Actions: Maliciously invoking internal administrative activities via Intent Redirection to alter settings or transfer funds.

Local Data Tampering: Modifying local SQLite database records via exported ContentProviders or backup injections.

Forged API Requests: Replaying or forging backend transactions using hijacked credentials with full user authorization.

Risk: Unauthorized State Changes & Transaction Forgery

AVAILABILITY

SEVERITY: MODERATE

Intent Fuzzing Crashes: Sending malformed extras or unparseable parcels causing NullPointerException or BadParcelableException.

Background Service DoS: Spamming exported Services to cause resource starvation and system battery drain.

Database Lockout: Corrupting SQLite journal files, rendering the application unusable upon subsequent launches.

Risk: App Denial of Service & Device Battery Drain

DEFENSIVE ENGINEERING

Defense Strategy: Multi-Layered Hardening

A Comprehensive Architectural Blueprint for Secure Android Apps

Layer 1: Manifest & Access

Explicit exported="false": Declare every internal Activity, Service, and Receiver as private by default.

Least Privilege: Audit and eliminate unnecessary `<uses-permission>` tags. Request runtime permissions just-in-time.

Signature Permissions: Gate inter-app endpoints using `protectionLevel="signature"`.

Disable Backups: Set `android:allowBackup="false"` to prevent ADB extraction.

```
<activity
    android:name=".InternalUI"
    android:exported="false" />
```

Layer 2: Intent Security & IPC

Mandate Explicit Intents: Use explicit class bindings for all internal application communication.

Component Whitelisting: Before forwarding nested Intents, verify `intent.component` matches allowed classes!

IntentSanitizer: Filter allowed actions, categories, and extras via AndroidX.

FLAG_IMMUTABLE: Always enforce on PendingIntents to prevent rewriting.

```
val safeIntent = IntentSanitizer.Builder()
    .allowComponent(allowedClass)
    .build().sanitize(untrustedIntent)
```

Layer 3: Storage Cryptography

Zero Plaintext at Rest: Never persist unencrypted auth tokens or credentials in local SQLite or SharedPreferences.

Deploy SQLCipher: Encrypt SQLite database pages with AES-256 transparent encryption.

EncryptedSharedPreferences: Encrypt key-value pairs using Google Jetpack Security.

Android Keystore: Store master keys in hardware TEE / Secure Element.

```
EncryptedSharedPreferences.create(
    context, "secure_prefs", masterKey,
    PrefKeyEnc.AES256_SIV, PrefValEnc.AES256_GCM)
```

PRACTICAL DEMONSTRATION PLAN

Final Demo: MITM Interception & SSL Pinning

Securing the Client-Server Transit Channel in a Controlled Lab

Phase 1: Baseline MITM (Vulnerable)

Android App → **Burp Proxy (CA Trusted)** → Backend Server

Vulnerable Setup: Test configuration trusts user-installed Certificate Authority (CA) certificates or default network trust.

Proxy Interception: Client traffic is routed through Burp Suite on port 8080; Burp generates a dynamic TLS cert for the backend.

Outcome in Lab: HTTPS traffic is decrypted in real time. Plaintext login credentials, chat messages, and API tokens are captured!

Result: Plaintext credentials & tokens captured in transit

Phase 2: Certificate Pinning (Hardened)

Android App → **[Pin Check] ✖ Burp Rejected**

Pinning Mechanism: Client validates server's public key SHA-256 hash using OkHttp `CertificatePinner` or `network_security_config.xml`.

Immediate Handshake Abort: When Burp presents its proxy certificate, the public key hash does NOT match the pinned hash.

Outcome in Lab: `SSLPeerUnverifiedException` is thrown; handshake is terminated before any HTTP payload is transmitted!

Result: TLS handshake terminated immediately • Zero data leak

AUTHORIZED EDUCATIONAL LAB METHODOLOGY

1. Develop simple chat app → 2. Route traffic via Burp Suite → 3. Capture & analyze baseline credentials → 4. Implement OkHttp CertificatePinner → 5. Empirically verify SSL handshake rejection.

EXECUTIVE SUMMARY

Key Takeaways & Conclusion

Core Architectural Principles for Secure Mobile Engineering

1. Sandboxing is Necessary, Not Sufficient: Linux UID process isolation protects internal files, but misconfigured component exposure (`android:exported="true"`) completely bypasses it.

2. Intents Form Trust Boundaries: Every Intent crossing an application boundary is untrusted input. Always validate actions, URIs, and allowlist nested Intent destinations.

3. Access Control ≠ Cryptographic Security: Sandbox permissions only control file access. Sensitive tokens in SQLite require encryption at rest (SQLCipher / Keystore) to survive device-level leaks.

4. Defense-in-Depth Across All Layers: Secure mobile systems require simultaneous hardening across the Manifest, IPC validation, local cryptographic storage, and network transit.

5. Theory Drives Practice: Our midterm theoretical findings establish the exact architectural foundation for our upcoming final laboratory demonstration with Burp Suite and SSL Pinning.

Thank you for your attention! We welcome any questions from the instructor and class.

ACADEMIC SOURCES

References & Industry Standards

Authoritative Literature, Frameworks & Specifications

[1] Google Android Developers (2024)

Permissions on Android & Protection Levels

Official Android Security Guides. https://developer.android.com/guide/topics/permissions/overview

Protection Levels & Runtime Permissions

[2] Google Android Developers (2024)

Intents and Intent Filters Documentation

Core Component IPC Architecture. https://developer.android.com/guide/components/intents-filters

IPC & Component Resolution Architecture

[3] Google Android Developers (2023)

Remediating Intent Redirection Vulnerability

Android Privacy & Security Risk Advisory. https://developer.android.com/privacy-and-security/risks/intent-redirection

Vulnerability Mechanics & Remediation Guidance

[4] OWASP Foundation (2023)

Mobile Application Security Verification Standard (MASVS v2.0)

Industry Benchmark for Mobile Security Architecture & Storage. https://mas.owasp.org/

MASVS-STORAGE & MASVS-IPC Verification Standard

[5] OWASP Foundation (2024)

Mobile Application Security Testing Guide (MASTG)

Technical Guide for Storage & IPC Security Verification (MASTG-STORAGE-0037). https://mas.owasp.org/

Testing Local Storage & SQLite Security Procedures

[6] NIST (2019)

Vetting the Security of Mobile Applications

NIST Special Publication 800-163 Rev. 1. https://doi.org/10.6028/NIST.SP.800-163r1

Federal Guidelines for Mobile Application Vetting